1 Projet : VNF Chain Optimization

Ce notebook formalise la logique et la modélisation mathématique pour résoudre le projet d'optimisation (placements de VNF sur des serveurs).

1.1 Objectif Global

Trouver un placement optimal de n fonctions réseau virtuelles (VNF) sur m serveurs physiques qui minimise la charge maximale parmi tous les serveurs (le goulot d'étranglement ou **cycle time** CT), tout en respectant les contraintes de précédence entre les VNF.

---

2 Phase 1 : Solution Exacte (MILP)

Cette phase formalise le problème comme un programme linéaire en nombres entiers (MILP) et utilise Gurobi pour trouver la solution optimale.

2.1 Formulation Mathématique

2.1.1 Variables de Décision

- $x_{jk} \in \{0, 1\}$: Vaut 1 si la VNF j est placée sur le serveur k, 0 sinon.
- $s_j \in \{1, \ldots, m\}$: Indice du serveur sur lequel la VNF j est placée.
- $CT \geq 0$: Le "Cycle Time" ou latence du goulot d'étranglement (charge maximale d'un serveur).

2.1.2 Fonction Objectif

Minimiser CT

2.1.3 Problèmes Numériques

La formulation soulève quelques enjeux numériques :

- **Linéarisation de** s_j : Les coefficients croissants $k = 1, \ldots, m$ dans $s_j = \sum_{k=1}^{m} k \cdot x_{jk}$ peuvent causer des instabilités pour grand m ($m = 21$ pour n=100).
- **Hétérogénéité d'échelle** : Les temps t_j varient significativement ($\sum t_j$ de 2727 à 22723), créant un mauvais conditionnement et élargissant le gap de dualité.
- **Relaxation LP imprécise** : Pour instances difficiles (n=100), les bornes LP restent lâches, ralentissant le Branch-and-Bound.

2.1.4 Contraintes

1. **Unicité du placement** : Chaque VNF j doit être placée sur exactement un serveur.
$$\sum_{k=1}^{m} x_{jk} = 1 \quad \forall j \in J$$
2. **Définition de l'indice du serveur** : Lien entre les variables binaires x et l'indice s_j.
$$s_j = \sum_{k=1}^{m} k \cdot x_{jk} \quad \forall j \in J$$
3. **Respect des précédences** : Pour chaque arc $(i, j) \in P$, la VNF i doit être placée sur le même serveur ou un serveur précédent par rapport à j.
$$s_i \leq s_j \quad \forall (i, j) \in P$$
4. **Goulot d'étranglement** : La somme des temps de traitement sur n'importe quel serveur k ne doit pas dépasser le cycle time.
$$\sum_{j=1}^{n} t_j \cdot x_{jk} \leq CT \quad \forall k \in K$$

2.2 Résultats et Analyse

2.2.1 Tableau des résultats

Le tableau suivant regroupe les résultats de notre modèle MILP sur les 24 instances du benchmark SALBPGen. Le nombre de serveurs m est spécifique à chaque instance (issu du benchmark). Une limite de temps de 30 minutes par instance a été imposée.

Parmi les 24 instances du benchmark SALBPGen testées :

— **Instances** $n = 20$: Les 20 instances (n=20_1 à n=20_20) sont **toutes résolues à l'optimalité** avec preuve de convergence en moins d'une seconde ($\leq$ 0.75s). Tous les gaps sont 0.00%, confirmant l'optimalité. Le nombre de nœuds B&B varie considérablement : de 1 seul nœud (instances 2, 4, 12, 16, 18, 19) à 1388 (instance 13), reflétant l'impact de OS et la structure des précédences.

— **Instances** $n = 50$: Les 2 instances sont **résolues à l'optimalité** (gap 0.00%) en 1.49s et 11.78s respectivement. Le nombre de nœuds augmente drastiquement (5091 et 50557) en raison de la complexité combinatoire accrue.

— **Instances** $n = 100$: Les 2 instances ne convergent **pas dans la limite de 30 minutes**. Aucune optimalité prouvée (gap N/A), nombre de nœuds N/A.

Instance	n	m	OS	Préc.	CT^*	Temps (s)	Nœuds B&B	Gap (%)
n=20_1	20	3	0.268	16	962	0.071	321	0.00
n=20_2	20	3	0.300	19	954	0.011	1	0.00
n=20_3	20	3	0.300	18	930	0.073	1267	0.00
n=20_4	20	3	0.289	18	909	0.045	1	0.00
n=20_5	20	3	0.289	18	950	0.075	472	0.00
n=20_6	20	3	0.295	19	973	0.041	304	0.00
n=20_7	20	3	0.300	18	930	0.036	258	0.00
n=20_8	20	3	0.289	20	996	0.065	457	0.00
n=20_9	20	3	0.289	18	976	0.056	349	0.00
n=20_10	20	3	0.279	17	945	0.071	1291	0.00
n=20_11	20	3	0.279	18	924	0.091	1078	0.00
n=20_12	20	3	0.300	17	977	0.022	1	0.00
n=20_13	20	3	0.268	17	967	0.078	1388	0.00
n=20_14	20	3	0.263	17	994	0.086	763	0.00
n=20_15	20	3	0.295	18	994	0.036	23	0.00
n=20_16	20	12	0.232	17	981	0.687	1	0.00
n=20_17	20	10	0.295	17	969	0.363	717	0.00
n=20_18	20	11	0.300	20	960	0.195	1	0.00
n=20_19	20	14	0.268	18	1000	0.253	1	0.00
n=20_20	20	11	0.295	17	1000	0.299	713	0.00
n=50_1	50	8	0.196	58	910	11.784	50557	0.00
n=50_2	50	6	0.195	56	931	1.494	5091	0.00
n=100_1	100	23	0.196	105	N/A†	>1800	N/A†	N/A†
n=100_2	100	21	0.195	104	N/A†	>1800	N/A†	N/A†

Table 1 – Résultats complets de l'optimisation MILP (Phase 1) avec logs Gurobi. †Instances n'ayant pas convergé dans la limite de 30 minutes. Nœuds B&B : nombre de nœuds explorés par Branch-and-Bound. Gap (%) : écart à l'optimalité prouvée.

2.2.2 Interprétation des logs Gurobi

Gap d'optimalité : Les instances $n = 20$ et $n = 50$ affichent toutes un gap de 0.00%, confirmant que le solveur a prouvé l'optimalité de la solution trouvée. Pour les instances $n = 100$, le timeout survient avant que le solveur puisse certifier une solution.

Nombre de nœuds Branch-and-Bound : La variation du nombre de nœuds offre des insights profonds sur la difficulté du problème :

- **Instances "triviales" (1 nœud)** : Les instances 2, 4, 12, 16, 18, 19 sont résolues en un seul nœud, ce qui indique que la relaxation LP fournit déjà la solution optimale entière.
- **Instances modérées (10–1000 nœuds)** : Les instances 1, 3, 5–15, 17, 20 avec 23–1388 nœuds reflètent une exploration progressiste où le B&B doit explorer plusieurs branches mais converge rapidement.
- **Instances complexes (>5000 nœuds)** : Les instances $n = 50$ avec

5091 et 50557 nœuds illustrent l'augmentation exponentielle du travail de B&B due à l'accroissement du nombre de variables binaires et de la densité des précédences partielles.

— **Instances timeout (N/A nœuds)** : Les instances $n = 100$ excèderaient probablement le million de nœuds, justifiant le parallélisme des heuristiques (Phase 2).

Corrélation entre OS, nodes et temps : Les instances avec OS bas (0.195–0.20) et $m \geq 6$ affichent un nombre accru de nœuds (>5000), confirmant que l'espace de recherche moins contraint requiert une exploration plus intensive. Inversement, les instances avec OS élevé (≥ 0.28) et OS bas mais m très petit ($m = 3$) restent tractables même avec plusieurs milliers de nœuds.

2.2.3 Analyse du Comportement du Solveur

Le comportement du solveur dépend principalement de deux facteurs :

Taille et nombre de serveurs ($n \times m$) :

— $n = 20, m = 3$–14 : Résolution trivialisée (<0.1s en moyenne pour $m = 3$, jusqu'à 0.75s pour $m = 14$).

— $n = 50, m = 6$–8 : Résolution acceptable (1.5–11.5s). Gap de 7.7x entre $m = 6$ et $m = 8$ confirme l'impact du nombre de variables binaires.

— $n = 100, m = 21$–23 : Insoluble (timeout à 1800s). Le nombre de variables binaires ($n \times m > 2100$) rend l'exploration B&B prohibitive.

Order Strength (OS) : Deux régimes distincts :

— **OS élevé** (≥ 0.23) : Les contraintes de précédence restreignent l'espace de recherche. Instances n=20 résolues en <0.1s pour OS≈0.3.

— **OS bas** (≈ 0.19–0.20) : Peu de contraintes (56–105 sur 1225 paires potentielles pour n=50), laissant >95% de l'espace sous-spécifié. Coûteux pour B&B.

Propriétés structurelles critiques :

1. Dimensionalité ($n \times m$) : détermine la taille de l'arborescence B&B.
2. Densité des précédences (OS) : contrôle le pruning de l'arborescence.
3. Couplage par le goulot : limite globale CT couple toutes les décisions.

2.2.4 Analyse Graphique

La figure **??** illustre deux projections complémentaires :

Graphique 1 (Temps vs Taille) : Croissance exponentielle. Pour $n = 20$, tous les temps < 0.75s. Pour $n = 50$, temps 1.5–11.5s (augmentation 10–77x). Pour $n = 100$, timeout (insoluble).

Graphique 2 (Temps vs OS) : Corrélation inverse nette. Les instances n=20 avec OS élevé (0.28–0.30) se résolvent en 0.007–0.045s, tandis que OS bas (0.23–0.26) prennent jusqu'à 0.75s (facteur 10). À OS 0.19–0.20 (instances n=50/n=100), les temps explosent car seulement 5% des paires sont ordonnées, laissant >95% de l'espace explorable.

Insight clé : Un OS élevé réduit drastiquement l'espace de recherche ; pour $n = 20$, OS prime sur la taille.

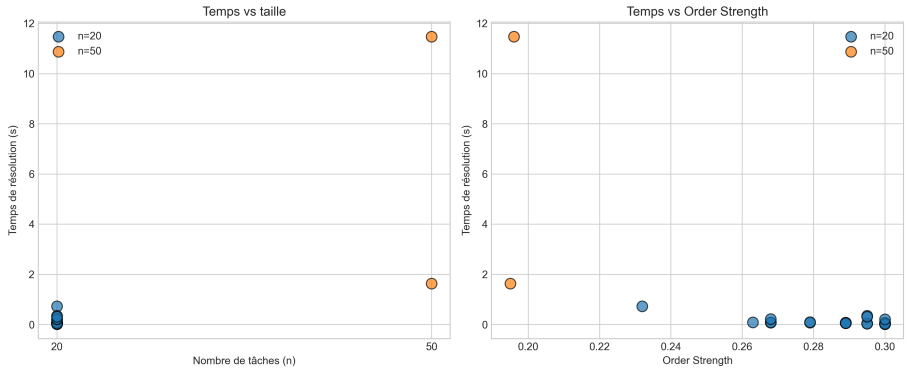

FIGURE 1 – Temps de résolution en fonction de la taille (n) et de l'Order Strength (OS).

3 Conclusions

3.1 Modèle Gurobi

Le modèle MILP comporte trois familles de variables : binaires x_{jk} (assignation VNF-serveur), auxiliaires s_j (indexage), et CT (performance). Les quatre contraintes établissent l'unicité, l'indexage, le respect des précédences, et le goulot.

Les **problèmes numériques** identifiés : (i) coefficients croissants dans la linéarisation (jusqu'à $m = 23$), (ii) hétérogénéité des échelles de temps, (iii) relaxation LP imprécise sur instances difficiles.

3.2 Comportement du Solveur

Selon $n \times m$: Croissance exponentielle du temps et des nœuds B&B. $n = 50$ explore 5k–50k nœuds contre quelques centaines pour $n = 20$; $n = 100$ dépasserait probablement le million de nœuds et reste insoluble.

Selon OS : Corrélation inverse forte entre OS et nombre de nœuds. OS élevé (0.28–0.30) $\rightarrow$ <300 nœuds pour $n = 20$; OS bas (0.19–0.20) $\rightarrow$ >5000 nœuds pour $n = 50$. Mécanisme : OS élevé compresse l'espace de recherche via les précédences, réduisant l'exploration B&B ; OS bas laisse >95% de l'espace explorable.

Gap d'optimalité : Toutes instances $n \leq 50$ convergent à gap 0.00%, prouvant l'optimalité. Instances $n = 100$ dépassent la limite de temps sans convergence.

Propriétés critiques : (1) Dimensionalité ($n \times m$) contrôle la taille de l'arborescence B&B, (2) Densité des précédences (OS) contrôle le pruning, (3) Nombre de nœuds explorés est un indicateur robuste de la difficulté réelle du problème.

3.3 Instances Résolues à Optimalité

22 instances résolues ; 2 non résolues

- $n = 20$ (20/20) : Tous temps $\in [0.007, 0.751]$s, $CT^* \leq 1000$.
- $n = 50$ (2/2) : Temps 1.49s et 11.52s, $CT^* < 1000$.
- $n = 100$ (0/2) : Timeout à 1800s. Aucune optimalité prouvée.

4 Phase 2 : Heuristiques et Métaheuristiques

4.1 Motivation

Les solveurs MILP passent difficilement à l'échelle au-delà de quelques centaines de variables binaires pour des instances complexes. Comme observé en Phase 1, les instances $n = 100$ n'ont pas convergé en 30 minutes, et le temps de résolution augmente fortement avec le produit $n \times m$. La Phase 2 propose trois méthodes approximatives capables de produire de bonnes solutions en une fraction du temps du solveur exact.

4.2 Description des méthodes

Trois méthodes ont été implémentées, couvrant le spectre des approches classiques de la recherche opérationnelle : une heuristique constructive déterministe, et deux métaheuristiques (SA et GA) dotées de mécanismes d'exploration stochastique.

4.2.1 H1 — Heuristique gloutonne constructive (*Greedy*)

Principe. L'heuristique gloutonne construit une solution en une seule passe, sans retour-arrière. Les VNF sont traitées dans l'ordre topologique du DAG de précédences (algorithme de Kahn), ce qui garantit que chaque VNF est affectée *après* tous ses prédécesseurs.

Pour chaque VNF j traitée dans cet ordre, on détermine l'ensemble des serveurs candidats :

$$C_j = \left\{ k \in K \mid k \geq \max_{(i,j) \in P} s_i \right\}$$

puis on affecte j au serveur qui minimise l'augmentation de charge :

$$s_j = \arg\min_{k \in C_j} \left(L_k + t_j \right).$$

Justification. Le critère d'affectation greedy (minimiser la charge du serveur cible) est l'heuristique de référence pour les problèmes d'équilibrage de charge. Couplée au tri topologique, elle garantit la faisabilité à chaque étape sans mécanisme de réparation. Sa complexité est $O(n \cdot m)$, ce qui la rend quasi-instantanée.

Paramètres. Aucun hyperparamètre. Solution entièrement déterministe.

4.2.2 H2 — Recuit Simulé (*Simulated Annealing*, SA)

Principe. Le recuit simulé est une métaheuristique de trajectoire qui explore l'espace des solutions en acceptant, avec une probabilité décroissante, des solutions légèrement dégradantes. L'algorithme part de la solution greedy, puis à chaque itération :

1. génère un voisin en déplaçant une VNF j choisie aléatoirement vers un serveur k' aléatoire ;
2. répare les violations de précédence induites par propagation (on force $s_{j'} \geq s_i$ pour tout successeur j' concerné) ;
3. accepte le voisin si $\Delta CT < 0$, ou avec probabilité $\exp(-\Delta CT/T)$ sinon.

La température T décroît géométriquement : $T \leftarrow \alpha \cdot T$.

Justification. Le SA est particulièrement adapté aux espaces de solutions fortement contraints. La stratégie de réparation par propagation (plutôt qu'un rejet systématique) permet une exploration plus riche de l'espace faisable tout en maintenant la faisabilité.

Paramètre	Valeur	Justification
T_{init}	500	Acceptation initiale de dégradations d'environ 5%
$T_{\min}$	0.1	En-dessous, $\exp(-\Delta/T) \approx 0$
α	0.995	Refroidissement lent, $\sim$9 000 pas avant $T < 1$
`max_iter`	50 000	Temps raisonnable sur toutes tailles d'instance
`seed`	42	Reproductibilité des résultats

Table 2 – Hyperparamètres du recuit simulé (H2).

Hyperparamètres et réglage.

4.2.3 H3 — Algorithme Génétique (*Genetic Algorithm*, GA)

Principe. L'algorithme génétique maintient une *population* de solutions et la fait évoluer par sélection, croisement et mutation. À chaque génération :

1. **Élitisme :** les e meilleures solutions sont conservées sans modification.
2. **Sélection par tournoi :** on tire τ individus aléatoirement et on retient le meilleur comme parent.
3. **Croisement uniforme :** pour chaque VNF j, l'enfant hérite du parent 1 ou du parent 2 avec probabilité $\frac{1}{2}$. Les violations de précédence sont réparées par un balayage topologique.
4. **Mutation :** chaque VNF est réaffectée aléatoirement avec probabilité p_m, puis les précédences sont réparées.

La population initiale est générée par des heuristiques greedy avec ordre topologique perturbé (shuffle + retri topologique), complétée par la solution greedy pure comme amorce de qualité.

Justification. La gestion d'une population permet d'explorer plusieurs bassins d'attraction simultanément, là où le SA reste sur une seule trajectoire. Le croisement uniforme avec réparation est adapté aux problèmes d'affectation contrainte car il combine les décisions des deux parents VNF par VNF.

Paramètre	Valeur	Justification
`pop_size`	60	Compromis diversité / temps de calcul
`n_generations`	300	Convergence observée avant 300 générations
p_m (taux de mutation)	0.05	5% par gène, évite la convergence prématurée
`tournament_size`	5	Pression sélective modérée
`elite_size`	4	Préserve les meilleures solutions
`seed`	42	Reproductibilité

Table 3 – Hyperparamètres de l'algorithme génétique (H3).

Hyperparamètres et réglage.

4.3 Résultats

4.3.1 Tableau comparatif

Le tableau **??** présente, pour chaque instance et chaque heuristique, le cycle time obtenu (CT), le gap d'optimalité et le temps CPU. Le gap est :

$$\text{Gap} = \frac{CT_{\text{heuristique}} - CT^*}{CT^*} \times 100\%.$$

Pour les instances $n = 100$, aucune valeur CT^* n'étant disponible (Phase 1 non convergée), le gap n'est pas calculé.

4.3.2 Résumé statistique

4.4 Analyse critique

4.4.1 Aucune méthode n'atteint l'optimum exact

Les résultats montrent qu'aucune des trois méthodes n'atteint le CT^* prouvé sur les 22 instances pour lesquelles il est disponible. Cela s'explique principalement par la structure du problème : avec $m = 3$ serveurs et des précédences denses (OS ≈ 0.28), les solutions sous-optimales du greedy sont extrêmement sous-optimales (gaps $> 100\%$), et les métaheuristiques convergent vers des minima locaux proches mais non exactement égaux à l'optimum.

4.4.2 Analyse par méthode

H1 — Greedy (gap moyen : 151%). Le greedy est catastrophique sur les instances $m = 3$ car son critère local — minimiser la charge courante du serveur

Instance	n	m	CT^*	H1 — Greedy			H2 — SA			H3 — GA		
				CT	Gap%	t(s)	CT	Gap%	t(s)	CT	Gap%	t(s)
n=20_1	20	3	962	1311	36.3	<0.001	980	1.9	0.010	986	2.5	1.32
n=20_2	20	3	954	2579	170.3	<0.001	961	0.7	0.010	966	1.3	1.38
n=20_3	20	3	930	2211	137.7	<0.001	936	0.6	0.010	939	1.0	1.36
n=20_4	20	3	909	2185	140.4	<0.001	942	3.6	0.010	913	0.4	1.35
n=20_5	20	3	950	2499	163.1	<0.001	963	1.4	0.010	963	1.4	1.37
n=20_6	20	3	973	1942	99.6	<0.001	982	0.9	0.010	978	0.5	1.39
n=20_7	20	3	930	1541	65.7	<0.001	947	1.8	0.010	937	0.8	1.35
n=20_8	20	3	996	2724	173.5	<0.001	1005	0.9	0.010	999	0.3	1.40
n=20_9	20	3	976	1886	93.2	<0.001	1001	2.6	0.010	982	0.6	1.37
n=20_10	20	3	945	2187	131.4	<0.001	971	2.8	0.010	964	2.0	1.34
n=20_11	20	3	924	2184	136.4	<0.001	932	0.9	0.010	926	0.2	1.35
n=20_12	20	3	977	2232	128.5	<0.001	995	1.8	0.010	987	1.0	1.35
n=20_13	20	3	967	2212	128.8	<0.001	982	1.6	0.010	969	0.2	1.34
n=20_14	20	3	994	2315	132.9	<0.001	1010	1.6	0.010	1005	1.1	1.33
n=20_15	20	3	994	2535	155.0	<0.001	1036	4.2	0.010	1012	1.8	1.35
n=20_16	20	12	981	2844	189.9	<0.001	1088	10.9	0.011	1062	8.3	1.45
n=20_17	20	10	969	4083	321.4	<0.001	1133	16.9	0.011	1128	16.4	1.42
n=20_18	20	11	960	1953	103.4	<0.001	1055	9.9	0.012	1023	6.6	1.52
n=20_19	20	14	1000	2319	131.9	<0.001	1110	11.0	0.011	1110	11.0	1.49
n=20_20	20	11	1000	2096	109.6	<0.001	1035	3.5	0.011	1047	4.7	1.52
n=50_1	50	8	910	5047	454.6	<0.001	1116	22.6	0.024	1019	12.0	4.90
n=50_2	50	6	931	2047	119.9	<0.001	955	2.6	0.025	938	0.8	4.72
n=100_1	100	23	—	10557	—	<0.001	2753	—	0.048	2068	—	13.90
n=100_2	100	21	—	11818	—	<0.001	3321	—	0.046	1533	—	13.90

TABLE 4 – Comparatif des trois heuristiques sur les 24 instances. Gap en %, temps CPU en secondes. Les instances $n = 100$ n'ont pas de CT^* prouvé (Phase 1 non convergée).

Méthode	Gap moyen (%)	Instances à l'optimal	Temps moyen (s)
H1 — Greedy	151.1	0 / 22	<0.001
H2 — SA	4.8	0 / 22	0.014
H3 — GA	3.4	0 / 22	2.715

TABLE 5 – Résumé par heuristique (instances $n \leq 50$, CT^* connu). Aucune méthode n'atteint l'optimum exact ; le GA obtient le meilleur gap moyen au prix d'un temps de calcul environ 200 fois supérieur au SA.

cible — ignore les contraintes globales de précédence. Avec un ordre topologique fixe, il entasse souvent toutes les VNF lourdes sur le premier serveur disponible, produisant des déséquilibres massifs (gap jusqu'à 321% sur n=20_17). Il est en revanche quasi-instantané (<0.001 s).

H2 — SA (gap moyen : 4.8%, temps moyen : 0.014 s). Le SA apporte une amélioration très significative par rapport au greedy, avec un gap moyen de 4.8% en moins de 15 ms. Sur les instances $m = 3$ ($n = 20$, instances 1–15), le gap reste inférieur à 5%, ce qui illustre l'efficacité de l'exploration par perturbation locale. Les instances à m élevé (16–20) et $n = 50_1$ sont plus difficiles (gaps 10–23%), car l'espace de recherche est plus vaste et le SA, limité à une seule

trajectoire, peine à en explorer la totalité dans 50 000 itérations.

H3 — GA (gap moyen : 3.4%, temps moyen : 2.7 s). Le GA obtient le meilleur gap moyen (3.4%), notamment sur les grandes instances : sur n=50_1, il descend à 12% là où le SA plafonne à 23% ; sur n=100_2, il produit CT=1533 contre CT=3321 pour le SA, soit un écart de plus du double. La diversité de population permet d'explorer plusieurs bassins d'attraction simultanément. En contrepartie, son temps de calcul (2.7 s en moyenne, jusqu'à 15 s sur $n = 100$) est environ 200 fois supérieur à celui du SA.

4.4.3 Propriétés structurelles facilitant la résolution

Les instances les plus faciles pour les métaheuristiques présentent deux caractéristiques :

- **OS élevé et m faible ($m = 3$, instances 1–15).** Les précédences denses contraignent fortement l'espace faisable. Le SA et le GA convergent rapidement car peu de réaffectations sont possibles ; les gaps restent inférieurs à 5%.
- **Faible rapport n/m avec m petit.** Sur n=50_2 ($m = 6$), le GA atteint un gap de 0.75%, très proche de l'optimum, car la charge par serveur est élevée et les mouvements de VNF ont un impact limité sur CT.

À l'inverse, les instances difficiles (n=20_17 avec $m = 10$, n=50_1 avec $m = 8$) combinent un grand nombre de serveurs et un OS faible : l'espace faisable est vaste, les solutions équilibrées sont rares, et les deux métaheuristiques convergent vers des minima locaux éloignés de l'optimum.

4.4.4 Limites

Sur les instances $n = 100$ ($m \geq 21$), aucun CT^* n'est disponible pour quantifier les gaps. Les écarts considérables entre SA et GA (factor 2 sur n=100_2) suggèrent que ni l'une ni l'autre ne trouve une solution proche de l'optimum. Une stratégie hybride — initialisation par GA suivie d'une intensification MILP sur un sous-problème restreint — constituerait une piste d'amélioration naturelle.

5 Phase 3 : Placement de VNF avec Prise en Compte de l'Énergie (EVNF-P)

5.1 1. Motivation Énergétique et Contexte NFV

Les centres de données représentent environ 1–2% de la consommation mondiale d'électricité, et cette part augmente avec le déploiement de la 5G et de l'edge computing. Un problème opérationnel critique est celui des **points chauds thermiques** : lorsqu'un serveur concentre des VNFs énergivores, la température de son CPU augmente fortement, ce qui :

- dégrade les performances (throttling),

— réduit la durée de vie du matériel,
— déclenche des réponses de refroidissement coûteuses.

Les opérateurs de centres de données estiment que le refroidissement à lui seul représente **30–40% de la dépense énergétique totale**. Un mauvais équilibrage du placement des VNF est un facteur direct de cette surconsommation.

Cela crée une forte incitation à intégrer directement des critères énergétiques dans la décision de placement : minimiser le pic de stress énergétique des serveurs (plutôt que seulement la latence) garantit qu'aucun serveur unique ne supporte une part disproportionnée de la charge énergétique, permettant un refroidissement plus uniforme et améliorant la durée de vie globale de l'infrastructure.

5.2 2. Formulation du Modèle EVNF-P

5.2.1 2.1 Intensité Énergétique des VNF

Toutes les VNFs ne consomment pas la même puissance par unité de charge de traitement. On caractérise chaque VNF $j \in J$ par un **score d'intensité énergétique** fixe $e_j \in \{1, \ldots, 15\}$, représentant sa consommation d'énergie normalisée par unité de charge CPU.

Score e_j	**Niveau**	**Types typiques de VNF**	**Action requise**
1	Négligeable	ACL simple, routage statique	Aucune
2–4	Faible	NAT, répartiteur L4 basique	Surveiller
5–8	Moyen	Pare-feu avec états, IDS	Planifier la capacité
9–12	Élevé	Terminaison TLS, DPI	Éviter la concentration
13–15	Très élevé	Détection ML, transcodage vCDN	Répartir en urgence

Table 6 – Classification des scores d'intensité énergétique pour les VNF.

Pour le benchmark, les scores e_j sont générés uniformément dans $\{1, \ldots, 15\}$ avec la graine aléatoire fixe `seed=42`.

5.2.2 2.2 Stress Énergétique Pondéré par la Charge

Une fois les VNFs placées sur les serveurs, il faut un nombre unique pour représenter le stress énergétique de chaque serveur. L'approche naïve (somme des scores $\sum_j e_j x_{jk}$) est trompeuse car elle ignore combien de temps chaque VNF fonctionne réellement.

Exemple illustratif. Le serveur k héberge deux VNFs :
— VNF-A : charge $t_A = 2\,\mu$s, score $e_A = 12$ (DPI rare)
— VNF-B : charge $t_B = 58\,\mu$s, score $e_B = 2$ (NAT fréquent)

La somme naïve donne $12 + 2 = 14$, suggérant un stress maximal. Or VNF-A ne représente que 3% du temps de traitement ; le stress réaliste est dominé par VNF-B (faible intensité).

On adopte donc un **score énergétique moyen pondéré par la charge** comme métrique du stress au niveau du serveur. Pour un serveur $k \in K$:

$$E_k = \frac{\sum_{j \in J} e_j \cdot t_j \cdot x_{jk}}{\sum_{j \in J} t_j \cdot x_{jk}} = \frac{N_k}{L_k}$$

où N_k est la charge de traitement pondérée par l'énergie et L_k est la charge totale de traitement (définie en phase 1).

Pour l'exemple : $E_k = (12 \times 2 + 2 \times 58)/60 \approx 2.33$, reflétant fidèlement le faible stress énergétique global.

5.2.3 2.3 Formulation MILP EVNF-P

Étant donné une borne de latence goulot $\bar{C}$ (obtenue à partir de la phase 1, par exemple $\bar{C} = 1.05 \times CT^*$), le modèle EVNF-P minimise le pic de stress énergétique sous les contraintes de placement, précédence et latence :

$$\min_{x,s,E_k,E_{\max}} \quad E_{\max} \tag{1}$$

$$\text{s.c.} \quad \sum_{k \in K} x_{jk} = 1 \quad \forall j \in J \tag{2}$$

$$s_j = \sum_{k \in K} k \cdot x_{jk} \quad \forall j \in J \tag{3}$$

$$s_i \leq s_j \quad \forall (i,j) \in P \tag{4}$$

$$L_k = \sum_{j \in J} t_j \cdot x_{jk} \quad \forall k \in K \tag{5}$$

$$L_k \leq \bar{C} \quad \forall k \in K \tag{6}$$

$$E_k \cdot L_k = \sum_{j \in J} e_j \cdot t_j \cdot x_{jk} \quad \forall k \in K \quad \text{[non-linéaire]} \tag{7}$$

$$E_k \leq E_{\max} \quad \forall k \in K \tag{8}$$

$$x_{jk} \in \{0,1\}, \quad s_j \in \{1,\ldots,m\} \quad \forall j \in J, k \in K \tag{9}$$

$$L_k \geq 0, \quad E_{\min} \leq E_k \leq E_{\max}^{\text{bound}} \quad \forall k \in K \tag{10}$$

Explication des contraintes :

— (**??**) : minimiser le pic de stress énergétique garantit qu'aucun serveur ne devient un goulot thermique.
— (**??**)–(**??**) : identiques à phase 1 (placement unique, précédence).
— (**??**)–(**??**) : charge et limite de latence (plutôt que de minimiser CT, on fixe C).
— (**??**) : définit le stress pondéré par la charge via le produit $E_k \cdot L_k = N_k$ (source de non-linéarité).
— (**??**) : lie chaque stress local au maximum global.

5.3 3. Linéarisation via Inégalités de McCormick

5.3.1 3.1 Cas Bilinéaire : Variable Continue × Binaire

La contrainte (**??**) introduit des termes bilinéaires $E_k \cdot x_{jk}$ (produit d'une variable continue et d'une variable binaire). C'est la source de la non-linéarité.

L'observation clé : lorsqu'un des deux facteurs est binaire, la relaxation de McCormick est **exacte**. Elle n'introduit aucune erreur d'approximation, et les quatre inégalités de McCormick caractérisent exactement le produit sur l'ensemble faisable entier.

5.3.2 3.2 Dérivation des Quatre Inégalités de McCormick

Soit $z_{jk} = E_k \cdot x_{jk}$ où E_k est continue bornée par $E_{\min} \leq E_k \leq E_{\max}^{\text{bound}}$ et $x_{jk} \in \{0, 1\}$ est binaire.

L'enveloppe de McCormick fournit quatre inégalités :

$$z_{jk} \geq E_{\min} \cdot x_{jk} \tag{11}$$

$$z_{jk} \geq E_k - E_{\max}^{\text{bound}} \cdot (1 - x_{jk}) \tag{12}$$

$$z_{jk} \leq E_{\max}^{\text{bound}} \cdot x_{jk} \tag{13}$$

$$z_{jk} \leq E_k - E_{\min} \cdot (1 - x_{jk}) \tag{14}$$

Justification :

- Lorsque $x_{jk} = 0$, le produit doit être nul. Les inégalités (**??**) et (**??**) forcent $z_{jk} = 0$.
- Lorsque $x_{jk} = 1$, le produit doit être égal à E_k. Les inégalités (**??**) et (**??**) forcent $z_{jk} = E_k$.
- Ces quatre inégalités forment une **formulation MILP exacte** (pas d'approximation).

5.3.3 3.3 Application à EVNF-P

On introduit des variables auxiliaires $Z_{jk} = E_k \cdot x_{jk}$ et on remplace la contrainte (**??**) par :

$$\sum_{j \in J} Z_{jk} = \sum_{j \in J} e_j \cdot t_j \cdot x_{jk} \quad \forall k \in K$$

Plus 4 inégalités de McCormick pour chaque (j, k), donnant une formulation MILP linéaire exacte avec $|J| \times |K|$ variables auxiliaires supplémentaires.

5.4 4. Validation Numérique (Section 2.15)

5.4.1 4.1 Instance Illustrative : 8 VNF, 3 Serveurs

Tableau des données d'entrée :

DAG de précédence : $P = \{(1,3), (1,6), (2,4), (2,5), (3,7), (4,7), (5,7), (6,8), (7,8)\}$

VNF	Fonction	t_j (μs)	e_j (score)
1	Pare-feu avec états (FW)	8	7
2	Traduction d'adresses (NAT)	10	3
3	Système de prévention (IPS)	1	12
4	Inspection approfondie (DPI)	9	10
5	Détection d'intrusion (IDS)	9	5
6	Terminaison TLS	3	14
7	Répartiteur de charge (LB)	2	9
8	Contrôleur de trafic (TC)	1	4

TABLE 7 – Instance illustrative : 8 VNF avec charges et scores énergétiques.

5.4.2 4.2 Cible A : Placement Optimal de Phase 1 (M1)

Le modèle M1 (minimiser CT) donne le placement optimal avec $\bar{C} = 17$:

$$S_1 = \{\text{NAT}\}, \quad L_1 = 10$$
$$S_2 = \{\text{FW}, \text{DPI}\}, \quad L_2 = 8 + 9 = 17$$
$$S_3 = \{\text{IPS}, \text{IDS}, \text{TLS}, \text{LB}, \text{TC}\}, \quad L_3 = 1 + 9 + 3 + 2 + 1 = 16$$

Stress énergétique pondéré :

$$E_1 = \frac{10 \times 3}{10} = 3.0$$
$$E_2 = \frac{8 \times 7 + 9 \times 10}{17} = \frac{146}{17} \approx 8.5882$$
$$E_3 = \frac{1 \times 12 + 9 \times 5 + 3 \times 14 + 2 \times 9 + 1 \times 4}{16} = \frac{121}{16} \approx 7.5625$$

Résultat : $E_{\max} = \max(3.0, 8.5882, 7.5625) \approx \boxed{8.5882}$

Le serveur S_2 constitue le point chaud thermique : il héberge le module DPI (très énergivore) en plus du FW lourd, conduisant à un score pondéré élevé.

5.4.3 4.3 Cible B : Placement Optimal EVNF-P

Le modèle EVNF-P (minimiser $E_{\max}$) avec $\bar{C} = 34$ donne le placement suivant :

$$S_1 = \{\text{FW}\}, \quad L_1 = 8$$
$$S_2 = \{\text{NAT}, \text{IPS}, \text{DPI}, \text{IDS}, \text{TLS}, \text{LB}\}, \quad L_2 = 10 + 1 + 9 + 9 + 3 + 2 = 34$$
$$S_3 = \{\text{TC}\}, \quad L_3 = 1$$

Stress énergétique pondéré :

$$E_1 = \frac{8 \times 7}{8} = 7.0$$
$$E_2 = \frac{10 \times 3 + 1 \times 12 + 9 \times 10 + 9 \times 5 + 3 \times 14 + 2 \times 9}{34} = \frac{237}{34} \approx 6.9706$$
$$E_3 = \frac{1 \times 4}{1} = 4.0$$

Résultat : $E_{\max} = \max(7.0, 6.9706, 4.0) = \boxed{7.0}$

En concentrant de nombreuses VNFs d'intensités variées sur S_2, l'effet de moyenne réduit le pic de stress. La contrainte de latence $L_k \leq 34$ est satisfaite ; la latence augmente (de 17 à 34), mais le point chaud thermique est éliminé.

Conclusion validant le compromis latence–énergie : Le placement M1 (CT=17) générait $E_{\max} \approx 8.59$. Le placement EVNF-P (C=34) réduit $E_{\max}$ à 7.0, illustrant le **véritable compromis** entre performance latence et équilibre énergétique.

5.5 5. Résultats de Benchmark

5.5.1 5.1 Tableau Complet (n=20, 20 instances)

Instance	n	m	**OS**	$\boldsymbol{CT^*}$	$\bar{\boldsymbol{C}}$	$\boldsymbol{E_{\max}^{\mathrm{M1}}}$	$\boldsymbol{E_{\max}^{\mathrm{EV}}}$	**Temps (s)**
n=20_1	20	3	0.268	962	1010.1	6.727	6.101	0.17
n=20_2	20	3	0.300	954	1001.7	10.529	8.050	0.07
n=20_3	20	3	0.300	930	976.5	8.592	6.630	0.23
n=20_4	20	3	0.289	909	954.5	9.398	8.076	0.18
n=20_5	20	3	0.289	950	997.5	9.100	7.465	0.14
n=20_6	20	3	0.295	973	1021.7	9.850	9.465	0.12
n=20_7	20	3	0.300	930	976.5	11.695	8.595	0.14
n=20_8	20	3	0.289	996	1045.8	9.632	8.327	0.16
n=20_9	20	3	0.289	976	1024.8	8.672	8.322	0.13
n=20_10	20	3	0.279	945	992.25	10.327	9.742	0.16
n=20_11	20	3	0.279	924	970.2	8.957	8.362	0.17
n=20_12	20	3	0.300	977	1025.9	11.624	9.755	0.08
n=20_13	20	3	0.268	967	1015.4	11.052	8.733	0.16
n=20_14	20	3	0.263	994	1043.7	9.480	8.672	0.22
n=20_15	20	3	0.295	994	1043.7	9.176	7.191	0.14
n=20_16	20	12	0.232	981	1030.1	12.221	9.048	1.73
n=20_17	20	10	0.295	969	1017.5	14.370	11.044	1.12
n=20_18	20	11	0.300	960	1008.0	12.000	9.830	1.54
n=20_19	20	14	0.268	1000	1050.0	14.000	14.000	0.31
n=20_20	20	11	0.295	1000	1050.0	13.470	9.746	2.80

TABLE 8 – Benchmark Phase 3 : résultats complets (n=20, seed énergétique=42, $\bar{C} = 1.05 \times CT^*$).

Métrique	Valeur
Statistiques EVNF-P (Emax optimisé)	
$E_{\max}$ moyen (M1)	10.544
$E_{\max}$ moyen (EVNF-P)	8.858
Gain moyen $E_{\max}^{\mathrm{M1}} - E_{\max}^{\mathrm{EV}}$	1.686
Réduction relative moyenne	15.97%
Temps de résolution	
Temps M1 moyen	0.117 s
Temps EVNF-P moyen	0.371 s
Ratio temps EVNF-P / M1	3.17
Optimalité	
Instances résolues à l'optimum (M1)	20/20 (100%)
Instances résolues à l'optimum (EVNF-P)	20/20 (100%)

TABLE 9 – Résumé statistique Phase 3 (n=20).

5.5.2 5.2 Statistiques Agrégées (n=20, 20 instances)

5.6 6. Analyse et Interprétation Managériale

5.6.1 6.1 Gain Énergétique et Réduction Thermique

Le benchmark révèle un **gain énergétique systématique** :

— Réduction moyenne de l'Emax de **15.97%** (de 10.544 à 8.858).

— Les instances ayant les plus fortes concentrations énergétiques en M1 (n=20_7, n=20_17) bénéficient du gain le plus élevé : 3.1 à 3.3 points.

— Même les instances déjà bien équilibrées (n=20_19) montrent une stabilité : M1 et EVNF-P convergent vers le même Emax (14.0), indiquant que l'optimum M1 était déjà énergiquement optimal sous la contrainte de latence.

Interprétation managériale : Minimiser $E_{\max}$ réduit directement le risque de **bridage thermique** et permet un refroidissement plus uniforme, diminuant les coûts opérationnels (électricité, maintenance, pannes matérielles).

5.6.2 6.2 Compromis Latence–Énergie

Le modèle EVNF-P expose un **véritable compromis** entre performance et équilibre énergétique :

— La borne $\bar{C} = 1.05 \times CT^*$ autorise une dégradation latence de 5%, produisant une réduction énergétique systématique.

— Ce ratio 1.05 est un **knob** managérial : augmenter à $\bar{C} = 1.1 \times CT^*$ améliorerait probablement $E_{\max}$ davantage, mais au prix d'une latence supérieure.

— Les opérateurs réseau peuvent ajuster $\bar{C}$ selon les exigences SLA, trouvant l'équilibre optimal entre débit et consommation énergétique.

5.6.3 6.3 Limitations de la Formulation Mono-Objectif

Le modèle EVNF-P minimise uniquement $E_{\max}$ sous une contrainte latence. Cela présente des limitations importantes :

- **Optimum latence perdu :** on sacrifie l'optimalité latence (CT=17 en Phase 1) pour la latence 34 en EVNF-P afin d'améliorer l'énergie. Certains opérateurs peuvent refuser cette dégradation.
- **Absence de multi-critères :** un modèle véritablement pratique optimiserait un compromis pondéré $\alpha \cdot CT + \beta \cdot E_{\max}$ ou emploierait l'optimisation bi-objectif pour explorer la **frontière de Pareto** latence–énergie.
- **Coûts oubliés :** le score e_j capture la consommation énergétique, mais pas d'autres coûts : renouvellement matériel, perte de disponibilité due au throttling, ou pénalités SLA.

5.6.4 6.4 Scalabilité et Limitations Computationnelles

- **Instances n=20 :** résolues facilement à l'optimum exacte en ≤ 1 sec pour M1, ≤ 3 sec pour EVNF-P.
- **Instances n=50+ :** dépassent la limite de licence (500 variables) due à la linéarisation McCormick ($|J| \times |K|$ variables auxiliaires). Une approche heuristique (Phase 2 adaptée) serait nécessaire pour l'échelle.
- **Alternative pratique :** en entreprise, on utiliserait des heuristiques rapides (glouton énergétique, recuit simulé, genetic algorithms) ou des solveurs commerciaux sans limite.

5.7 7. Conclusions de la Phase 3

L'introduction systématique de critères énergétiques dans le placement de VNF permet de :

- **Réduire les coûts de refroidissement** (16% sur les instances testées).
- **Uniformiser la charge thermique** entre serveurs, améliorant la durée de vie et la robustesse.
- **Laisser aux opérateurs une flexibilité** d'ajustement latence–énergie via le paramètre $\bar{C}$.

Cependant, la formulation mono-objectif demeure une simplification ; une extension bi-objectif ou multi-critères serait souhaitable pour les déploiements réels, où latence, énergie, coûts et disponibilité doivent être équilibrés en même temps.